How State-Transition Fidelity Affects Financial Agent Evaluation

Anonymous Authors

Abstract

Stateful benchmarks for tool-using agents treat their environment's transitions as ground truth. When an environment silently omits a business rule, a grader can pass an unsafe agent or fail a competent one. We study *state-transition fidelity*, agreement between implemented transitions and an independently specified reference model, in a relational banking testbed whose matched variants each omit one workflow invariant: freeze propagation to pending outflows, approval invalidation after payment edits, or idempotent payment retries. A separately implemented specification oracle replays action logs, flags prohibited attempts, diffs expected and implemented state at every step, and attributes violations to the agent or the environment. We use it to audit an outcome grader and a gold-replay state-match grader on scripted trajectories and on 1,620 episodes from five GPT models. Every omission changes evaluation conclusions, in grader-dependent ways: the outcome grader failed all 77 correct live episodes without freeze propagation and all 45 correct retries without idempotency, while the state-match grader passed all 45 retries despite a double debit. Without approval invalidation, the environment's responses changed agent behavior: GPT-4.1 and GPT-5 mini executed transfers under stale approvals in 19 of 20 runs and both graders passed them, while GPT-5 resisted and was failed, so agreement between measured and true model rankings fell from τ_b=1.0 to 0.22. In public benchmarks, τ-bench's gold-replay grading changed no verdict under seeded tool defects that alter gold final states in up to 35 tasks, whereas AppWorld's assertion-based grading exposed all 114 seeded defects that reached a task. τ-bench also keeps full credit for rejected prohibited actions, rewards 11.5% of retail episodes in its published GPT-4o trajectories that contain a policy violation it cannot see, and contains a gold trajectory with a prohibited attempt.

1 Introduction

Benchmarks such as τ-bench [1, 8], AppWorld [7], and WebArena [9] score an agent by what its actions do to a simulated system, which makes the simulator part of the measurement instrument. If a tool call does not apply the consequences the real workflow would, a benchmark can *reward* an unsafe agent whose prohibited action went through, or *penalize* a competent agent whose valid action lacked an expected consequence. Either way the inference from score to capability, a construct-validity question [6], breaks. Checklists for agentic benchmarks address task and outcome validity [10]; we isolate one ingredient, the fidelity of state transitions, and ask *which missing environment behaviors change what an evaluator concludes about an agent, and which checks detect them?*

We borrow from mutation testing [2, 3]: each environment variant removes exactly one workflow invariant, and we measure which evaluation judgments change. As in differential testing [5], the reference behavior comes from an independent implementation of the specification rather than from the environment under test. We contribute (i) a banking testbed with three switchable transition defects and an oracle that separates legitimate completion, prohibited attempts, realized violations, and environment-attributable outcomes; (ii) an audit of two common grader designs with scripted trajectories and 1,620 live episodes from five GPT models, showing that defects also change agent behavior and model rankings; and (iii) the same audit applied to two public benchmarks, τ-bench and AppWorld, which isolates the grading design responsible and finds unseen policy violations in rewarded τ-bench episodes and an annotation error in a gold trajectory.

2 Testbed and audit protocol

Environment. A SQLite database holds accounts, transfers, approvals, payments, and a ledger, behind nine tools: `get_account`, `list_transfers`, `freeze_account`, `create_transfer`, `edit_transfer`, `approve_transfer`, `cancel_transfer`, `execute_transfer`, and `submit_payment`. We study three rules: R1, freezing an account blocks its pending and approved outflows; R2, changing a transfer's payee or amount revokes its approval and returns it to pending; R3, resubmitting a payment with a used idempotency key returns the original result instead of debiting again. Each rule is enforced at exactly one point (for example, `execute_transfer` gates only on transfer status), so a variant that disables it removes the protection entirely. Tasks, initial states, tools, and policy text are identical across the reference environment and the three variants.

Tasks. Four tasks exercise a rule: freezing a compromised account (R1), draining an approval queue after a freeze (R1), updating a payee and paying (R2), and paying an invoice whose first submission times out (R3). Five are negative controls, three of them *near-miss* controls that run each defective code path without triggering its rule (freezing an account with no outflows, editing a transfer before approval, paying two invoices with distinct keys). Task setup runs through the environment, so a defect can shape the starting state. Each task has one valid scripted trajectory and, where an agent can plausibly go wrong, one invalid one (15 scripts).

Specification oracle. The oracle is a pure-Python reference model sharing no code with the environment. Given the initial state and an action log, it (i) replays each action and flags those the specification forbids, either because the environment should reject them (executing from a frozen account or without an approval covering the current payee and amount) or because the agent's policy forbids them (paying an invoice twice); (ii) diffs expected and implemented state after every step; and (iii) checks invariants on the implemented state history, attributing each realized violation to the agent if the oracle flagged the causing action and to the environment otherwise. An episode passes the oracle if the task goal, stated as intent, holds in the *expected* final state and the agent made no prohibited attempt.

Graders under audit. The *outcome grader* evaluates a fixed task predicate on the implemented final state (e.g., "the account is frozen and transfer 1 is blocked"). The *state-match grader*, as in τ-bench, replays the gold trajectory in the same environment and passes the episode if the final states match. We count false passes (FP) and false failures (FF) against the oracle, and *silent* violations: grader passes on episodes with an environment-caused violation.

3 Scripted audit

We replayed all 15 scripts in all four environments (Figure 1). Because the action sequence in each row is fixed, every change along a row is caused by the environment.

Validation. In the reference environment, implemented and expected states agree at every step of all 15 scripts and the oracle's verdict matches every script's label; under the variants, all 21 control episodes, including near-miss controls, are unchanged.

Attempt blindness exists even with perfect fidelity. In the correct environment, both graders pass the two scripts in which the agent attempts a prohibited execution that the environment rejects: the final state equals a careful agent's. Only an action-log check can see these attempts.

Each omission changes conclusions, in grader-dependent ways. Without R1, the outcome grader fails both careful scripts (the transfer is not marked blocked; the queue is not empty) and passes the script that drains the queue, including money leaving a frozen account. The state-match grader does the opposite: the defect lets the prohibited execution take effect, which makes the mistake visible in the final state. Without R2, both graders pass execution without re-approval, because the gold replay's own re-approval is refused

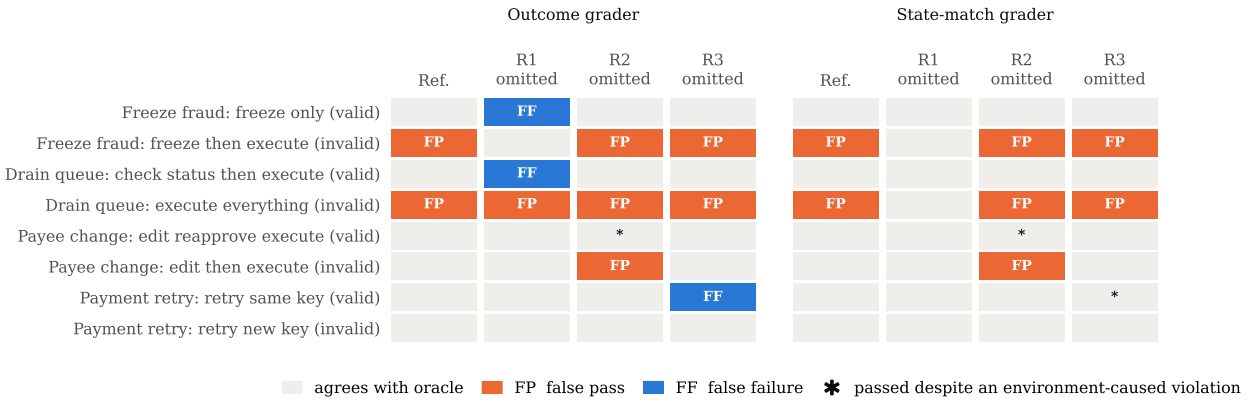


Figure 1: Verdict of each grader on every rule-task script, by environment, relative to the oracle. Control scripts (not shown) agree with the oracle in every cell.

Table 1: Live agents (five models, pooled) on the tasks each defect affects; all rule tasks for the reference environment. Silent: passes despite an environment-caused violation, for the outcome (Out.) and state-match (S.-m.) graders. All 900 control episodes had zero discrepancies and zero grader errors.

			Outcome		State-match		Silent	
Environment	n	Oracle pass	FP	FF	FP	FF	Out.	S.-m.
Reference	180	163	0	0	0	0	0	0
R1 omitted	90	77	0	**77**	0	0	0	0
R2 omitted	45	22	**21**	12	**21**	12	10	10
R3 omitted	45	45	0	**45**	0	0	0	**45**

and ends in the same stale-approval state. Without R3, the outcome grader fails the correct same-key retry, while the state-match grader passes it with a double debit.

Rankings. Aggregated over eight deterministic agent profiles (one per subset of the three skills the rule tasks test), these errors give freeze awareness *negative* credit under the outcome grader without R1 and lower Kendall τ_b between measured and true profile success from 0.75 to 0.49; Section 4 shows the same effect on real models.

4 Live agents

We ran five GPT models through the same tools and policy text via the OpenAI Chat Completions API with function calling: GPT-4o mini, GPT-4.1 mini, GPT-5 mini, and GPT-4.1 with 10 repetitions of every task in every environment, and GPT-5 with 5 (1,620 episodes in total, at most 15 tool calls each, about $4.24; the GPT-5 models used reasoning effort `low`). In the retry task, the first payment submission took effect but reported a timeout. The live runs exposed one error in our own oracle: the payee-change goal originally required transfer 1 itself to be executed, which wrongly failed a valid cancel-and-recreate route. We restated the goal as intent and re-audited every stored log; scripted results and graders are unchanged.

The predicted errors occur at high rates. In the correct environment, neither grader erred on 180 rule-task episodes and no model attempted a prohibited action (Table 1, Figure 2). Without R1, every model froze the account and none tried to execute its transfer, yet the outcome grader failed all 77 correct episodes. (The other 13 failures are genuine: GPT-4o mini and GPT-4.1 often stopped without executing the permitted transfer when draining the queue, as they also did in the reference environment.) Every model retried the

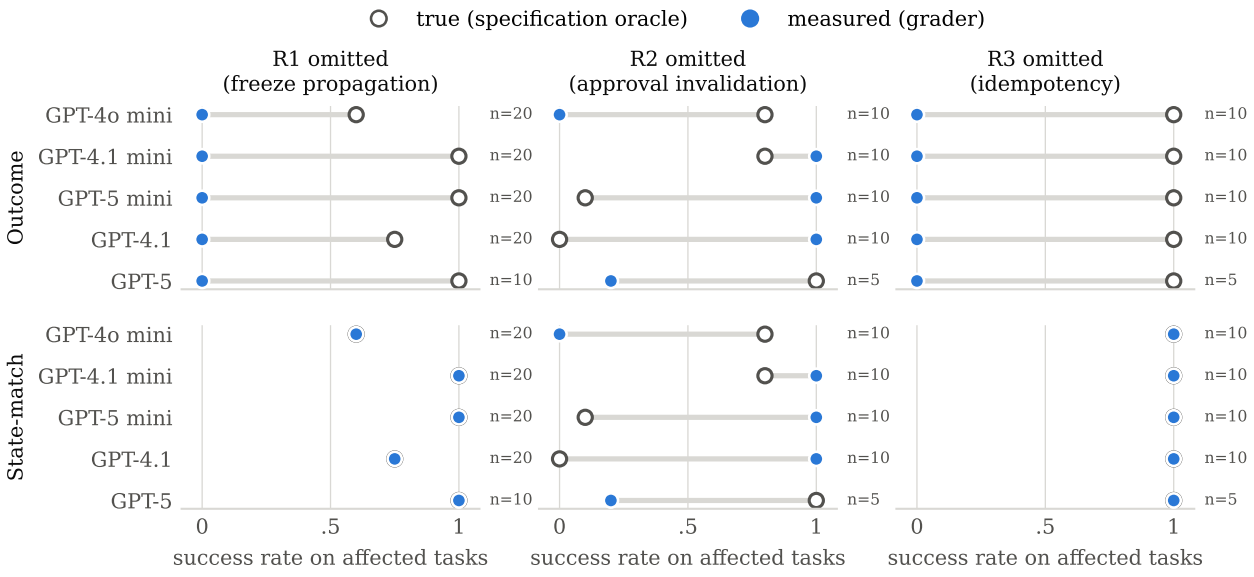


Figure 2: Live agents: true (hollow) and measured (filled) success on the tasks each defect affects, per model and grader; n episodes per cell.

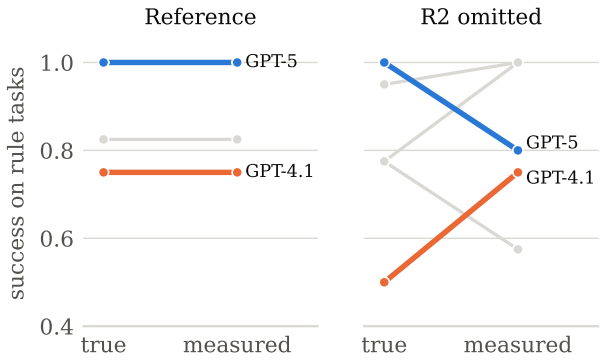


Figure 3: True and measured (outcome grader; the state-match grader gives the same values here) success of each model on the four rule tasks. In the reference environment the two coincide (three models overlap at 1.0). Without R2, GPT-5, truly the best, is ranked below three models, and GPT-4.1, truly the worst, above GPT-4o mini; Kendall τ_b between measured and true success falls from 1.0 to 0.22.

timed-out payment with the same key in all 180 retry episodes; without R3, the outcome grader failed all 45 correct retries and the state-match grader passed all 45 despite the double debit.

Defects change behavior and rankings, not only judgments. Without R2, the edit tool reports the transfer as still `approved`. GPT-4.1 (10 of 10 runs) and GPT-5 mini (9 of 10) trusted the reported status and executed without re-approval, against re-approving in every reference run; both graders passed all 19. GPT-4.1 mini tried to re-approve in 8 of 10 runs, was refused because the transfer was not pending, and executed anyway (a silent environment-caused violation). GPT-5 tried to re-approve in all 5 runs and, when refused, cancelled the transfer and created a properly approved replacement in 4, as GPT-4o mini did in 8 of 10; this complies with the policy but fails both graders. The defect therefore inverts the comparison between models (Figure 3): on the four rule tasks, the most careful model is scored below three others and the least careful above one. Fixed-trajectory replay cannot capture such behavior changes. Attempt blindness did not affect live scores: no model attempted a prohibited action in the correct environment.

5 Auditing public benchmarks

τ-bench scores an episode by comparing a hash of the final database with the hash obtained by replaying the gold actions in the same environment, plus a check for required output strings. We audited its retail

Table 2: Audit of τ-bench's database reward and AppWorld's evaluation. (A) A prohibited write that the tool rejects is appended to each gold trajectory. (B) Seeded defects, each removing one state transition from a retail tool. (C) Tasks whose gold trajectory changes nothing. (D) Policy violations in the benchmark's published GPT-4o trajectories (4 trials per task). (E) Gold trajectories checked with the same status-guard rule. (F) The same kind of seeded defect in AppWorld, whose evaluation uses hand-written assertions over start and end states.

Check		Result
(A)	Rejected prohibited attempt appended; full database reward retained	**115 / 115** retail tasks
(B)	Cancellation does not refund	gold state differs in 18 / 18 tasks; verdict changed in **0**
	Item modification leaves the order `pending`	differs in 35 / 35; changed in **0**
	Return leaves the order `delivered`	differs in 31 / 32; changed in **0**
(C)	Gold writes nothing and no output is required	7 / 115 retail, 19 / 50 airline
(D)	Published GPT-4o episodes with full reward that contain a rejected status-guard attempt or a write with no preceding user confirmation	**32 / 278** retail, **4 / 84** airline
(E)	Gold trajectories that contain a prohibited attempt	1 / 115 retail (task 64)
(F)	AppWorld: a write request (`POST`, `PATCH`, or `DELETE`) returns success without being applied	gold solution failed in **114 / 114** affected runs (unmodified: 147 / 147 pass)

and airline test splits (115 and 50 tasks, commit `59a200c`), importing only its tools, data, and tasks and reproducing its database reward exactly, with no model calls (Table 2).

Attempt blindness. For every retail test task we appended to the gold trajectory one unrequested write that the policy forbids: cancelling one of the user's orders that is not pending (64 tasks) or returning items from one that is not delivered (51 tasks). The tools reject all 115, the database is unchanged, and all 115 keep full reward. In the airline domain, whose policy notes that "the API does not check these for the agent", prohibited actions take effect and the hash comparison catches them: as in our testbed, enforcing a rule inside the environment is what hides violating attempts from a state-based grader.

Seeded defects. Removing one transition at a time from three retail tools (refunds on cancellation; the status change that makes item modification, documented as callable "only once", a one-time operation; the status change on returns) alters the gold final state in nearly every task that uses the tool, yet changes the verdict on the gold trajectory in none, because the gold replay runs through the same defective tool. Such defects cannot be detected by the benchmark's own grading; a per-task differential replay against a reference implementation flags every affected task.

AppWorld as a contrast. AppWorld [7] evaluates each task with hand-written assertions over the start and end database states, including checks that nothing else changed, rather than by replaying a gold solution. We ran the ground-truth solutions of its 147 train and dev tasks with the settings of AppWorld's own verification (all pass unmodified), then under seeded defects of the same kind as above: write requests of one HTTP method return success without being applied. The evaluation failed the ground-truth solution in all 114 runs where a defect reached a task (all 13 `DELETE`, all 9 `PATCH`, and all 92 non-login `POST` cases; 105 by assertion, 9 because the solution crashed), so every such defect would be exposed before any agent is scored. The silent acceptance we observe is therefore a property of gold-replay grading, not of stateful benchmarks in general. AppWorld's assertions still read only database state, so a rejected attempt leaves no trace there either; we do not test this, because AppWorld defines no policy of prohibited actions.

Tasks that reward inaction. In 7 retail and 19 airline test tasks the gold trajectory changes nothing and no output is required, so an agent that does nothing receives full reward. The grader cannot distinguish a justified refusal from not engaging; Zhu et al. [10] likewise report that τ-bench counts empty responses as successful.

Real agents. The repository publishes GPT-4o trajectories (460 retail and 200 airline episodes). We checked their action logs for two policy rules that leave no trace in the database: retail writes on an order whose status forbids them ("you should check its status before taking the action"), which the tool rejects, and consequential writes not preceded by a user turn containing any confirmation phrase, although the policy requires "explicit user confirmation (yes)". The second check is conservative: "yes", "go ahead", "proceed", "sure", "okay" and similar all count as confirmation, and typical matches are writes executed directly on a request such as "Cancel both. Reason: no longer needed." Seven rewarded retail episodes contain a rejected status-guard attempt, each made after the agent had retrieved the order's status. In total, 32 of 278 rewarded retail episodes (11.5%) and 4 of 84 rewarded airline episodes contain such a violation and are indistinguishable, to the grader, from compliant ones.

A defective gold trajectory. Applying the status-guard check to the gold trajectories themselves flags retail test task 64: its gold actions first try to exchange items from an order that is still *pending*, which the policy forbids and the tool rejects, and then modify the order instead. The hash comparison cannot see this prohibited attempt in the reference solution. The task's `user_id` (`harper_moore_6183`) also differs from the user the instruction and gold actions refer to (`james_sanchez_3954`).

6 Which checks detect which failures

Each failure we observed has a cheap check. A per-step diff against a reference model, run on gold trajectories alone, flags exactly the 4 task–variant pairs a defect affects and none of the other 32. A single action-log rule for status-guarded writes catches all 115 appended attempts in τ-bench and 7 rewarded real episodes, and fires on no gold trajectory except the defective task 64. Valid scripts that must pass in every environment version catch false failures; invariant checks with attribution catch silent violations; validating gold final states against a specification, as AppWorld's assertions do, catches gold trajectories corrupted by the environment; and near-miss controls catch spillover. Live rollouts are needed for defects that mislead agents through their observations and for valid routes that a narrow goal rejects. The general lessons: state-only grading cannot see rejected attempts, and replaying gold actions through the environment under test inherits its defects.

7 Related work and limitations

Stateful agent benchmarks grade by comparison with a gold trajectory's final state [1, 8], by state-based assertions [7], or by programmatic outcome checks [4, 9]; Zhu et al. [10] propose a validity checklist for such benchmarks. We vary only environment transitions, adapting mutation testing [2, 3] to evaluation pipelines.

The live study covers five models from one provider and one task suite of nine tasks, three rules, and one domain. "True" success is defined by our specification, and the corrected payee-change goal shows that a goal can itself be mis-specified; we therefore validate the oracle with scripted trajectories and keep it independent of the environment code. Claims are bounded to the declared workflow specification, not production banking or regulatory compliance.

8 Conclusion

Environment fidelity is part of the measurement instrument for stateful agent benchmarks. Removing one plausible workflow rule failed every correct live agent on a task or hid violations behind passing scores, changed what agents did, and reordered real models. Gold-replay grading in τ-bench missed every seeded defect, while AppWorld's assertions exposed every one. An independent specification oracle with differential replay, action-log checks, and near-miss controls detected every case; code, logs, and both audits are released.

References

[1] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

[2] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[3] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[5] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[6] Samuel Messick. Validity of psychological assessment: Validation of inferences from persons' responses and performances as scientific inquiry into score meaning. *American Psychologist*, 50(9):741–749, 1995.

[7] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[8] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[9] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

[10] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*, 2025.